第十三讲教学设计：三层架构与DAO模式重构 (20分钟)

课程信息

- **课程名称**: Java Web应用开发 (Java Web Application Development)
- **教学方式**: 双语教学（英文授课，关键术语中英文对照）
- **教学主题**: 三层架构与DAO模式重构（Servlet–Service–DAO 分层落地）
- **授课时长**: 20分钟
- **适用对象**: 已掌握Servlet/JSP、JDBC、会话管理与MVC基础的学生

一、教学目标

知识目标

1. 理解Web应用三层架构（Three-Tier Architecture: 表示层Presentation/Controller、业务层Business/Service、数据访问层Data Access/DAO）的职责边界与依赖方向。
2. 掌握DAO模式（Data Access Object Pattern）的核心思想：以接口（Interface）抽象数据访问；实现类屏蔽底层JDBC细节。
3. 掌握将JDBC访问代码从Servlet中抽离到DAO，并在Service中编排业务逻辑（Business Logic）与事务边界（Transaction Boundary）的基本方法。
4. 了解实体模型（Entity Model: JavaBean/POJO）的映射要点与包结构（Package Structure）组织规范。

能力目标

1. 能将“登录/列表查询”等功能从“控制器直连数据库”的写法，重构为“Controller→Service→DAO”的分层代码。
2. 能编写 `DAO接口 + JDBC实现` ，并在Service中合理处理异常与资源释放。
3. 能设计清晰的包结构与命名规范，提高可维护性与可测试性。

素质目标

1. 建立“关注点分离、分层解耦”的工程化思维。
2. 强化代码整洁度与可读性意识，避免“意大利面条式”代码。
3. **[思政融合]** 借“分工协作”的城市治理类比，强调职责清晰与协同配合对系统稳定与社会效率的意义。

二、教学重点与难点

教学重点

- 三层职责与依赖方向：Controller仅协调流程，Service聚合业务，DAO专注数据访问。
- DAO接口抽象与JDBC实现分离；Service中编排多个DAO调用。
- 包结构与命名规范，减少耦合。

教学难点

- 错误处理与异常转换（ `SQLException` 向业务异常的转换）。
- 资源管理位置（连接创建/关闭的职责归属）。
- 分层后数据对象（PO/DTO/VO）边界与最小必要暴露。

三、教学内容与时间安排

第一部分：问题引入——“控制器直连数据库”的隐患 (4分钟)

内容要点

1. 展示Servlet中混杂业务与JDBC代码的示例，指出问题：难测、难复用、难维护。
2. 思考：如何把“请求处理、业务编排、数据访问”解耦到不同职责层？
3. 引出三层架构与DAO模式。

教学方法

- 反例对比 + 问题驱动。

第二部分：新知讲解——分层落地与DAO模式 (9分钟)

内容要点

1. 包结构建议：

```
/src/com/demo
    /controller   // Servlet 控制器
    /service      // 业务服务（编排/校验/事务边界）
    /dao          // DAO接口与JDBC实现
    /model        // 实体模型（POJO/JavaBean）
    /util         // 工具类（DBUtil等）
/webapp/jsp
```

2. 实体模型（以User为例）：

```java
public class User {
    private Long id;
    private String username;
    private String passwordHash;
    private String role;
    // getters/setters
}
```

3. DAO接口与JDBC实现：

```java
// UserDao.java
public interface UserDao {
    User findByUsername(Connection conn, String username) throws SQLException;
}

// JdbcUserDao.java
public class JdbcUserDao implements UserDao {
    @Override
    public User findByUsername(Connection conn, String username) throws
SQLException {
```

```
        String sql = "SELECT id, username, password_hash, role FROM t_user WHERE
username = ?";
        try (PreparedStatement ps = conn.prepareStatement(sql)) {
            ps.setString(1, username);
            try (ResultSet rs = ps.executeQuery()) {
                if (rs.next()) {
                    User u = new User();
                    u.setId(rs.getLong("id"));
                    u.setUsername(rs.getString("username"));
                    u.setPasswordHash(rs.getString("password_hash"));
                    u.setRole(rs.getString("role"));
                    return u;
                }
                return null;
            }
        }
    }
}
```

4. DB工具（可沿用第六讲思路）：

```
public class DBUtil {
    public static Connection getConnection() throws SQLException {
        // 生产建议使用连接池；此处为演示
        return DriverManager.getConnection(URL, USER, PWD);
    }
    public static void closeQuietly(AutoCloseable c) {
        if (c != null) try { c.close(); } catch (Exception ignore) {}
    }
}
```

5. Service编排与异常转换：

```
public class UserService {
    private final UserDao userDao = new JdbcUserDao();

    public User login(String username, String plainPassword) {
        try (Connection conn = DBUtil.getConnection()) {
            User u = userDao.findByUsername(conn, username);
            if (u != null && PasswordUtil.matches(plainPassword,
u.getPasswordHash())) {
                return u;
            }
            return null;
        } catch (SQLException e) {
            throw new RuntimeException("数据访问失败", e);
```

```
        }
    }
}
```

6. Controller最小化：仅协调并分发到视图。

```
@WebServlet("/login")
public class LoginServlet extends HttpServlet {
    private final UserService userService = new UserService();
    @Override protected void doPost(HttpServletRequest req, HttpServletResponse
resp)
            throws IOException, ServletException {
        String username = req.getParameter("username");
        String pwd = req.getParameter("password");
        User u = userService.login(username, pwd);
        if (u != null) {
            req.getSession().setAttribute("user", u);
            req.setAttribute("userInfo", u);
            req.getRequestDispatcher("/jsp/welcome.jsp").forward(req, resp);
        } else {
            req.setAttribute("error", "用户名或密码错误");
            req.getRequestDispatcher("/jsp/login.jsp").forward(req, resp);
        }
    }
}
```

教学方法

- 代码走查 + 层次结构图 + 小练习（识别每段代码应放哪一层）。

第三部分：实战——把功能“拆下楼层、各归其位”（5分钟）

内容要点

1. 选取“用户登录/图书列表”任一功能，将JDBC从Servlet移出至DAO；
2. 在Service中聚合多个DAO调用（如分页查询需 `count+list` ）；
3. Servlet仅负责参数收集、调用Service、设置作用域并转发至JSP展示。

教学方法

- 现场重构 + 功能回归测试（验证页面行为不变、结构更清晰）。

第四部分：课堂总结与展望（2分钟）

总结要点

1. 三层架构实现关注点分离；DAO屏蔽数据访问细节，Service承载业务编排。
2. 分层使测试更容易、代码更稳定、演进更从容。

展望（承上启下）

- 下一讲将把“跨DAO的一致性”落到实处：数据库事务与连接池的工程化使用。

四、教学方法与手段

教学方法

1. **问题驱动 (Problem-Driven)**: 从控制器直连数据库的隐患引入
2. **代码走查 (Code Walkthrough)**: 逐层讲解DAO/Service/Controller
3. **现场重构 (Live Refactoring)**: 将混杂代码重构为三层结构
4. **结构图示 (Diagram)**: 展示包结构和依赖关系
5. **双语教学法 (Bilingual Teaching)**:
    - 课堂采用英文授课，关键技术术语提供中英文对照
    - 重要概念（Three-Tier Architecture, DAO Pattern, Business Logic, Transaction Boundary）首次出现时给出英文表达和中文释义
    - 鼓励学生使用英文术语描述分层架构
    - 代码注释和变量命名使用规范英文

教学手段

- IDE (IntelliJ IDEA/Eclipse)
- Tomcat服务器 (Tomcat Server)
- 数据库客户端 (Database Client: Navicat/DBeaver)
- PPT/Marp课件 (Presentation Slides)

五、教学评价

- **过程性评价**: 能否识别并说明一段代码在三层中的归属；能否写出 `UserDao` 接口与JDBC实现。
- **终结性评价**: 能否将一个“控制器直连数据库”的功能重构为三层结构并保持功能等价。

六、课后拓展

思考题

1. 为什么建议DAO返回领域对象（如 `User` ）而不是 `ResultSet` ？这对解耦有什么帮助？
2. DAO层应否感知事务？在不引入框架的情况下你会如何划分职责？

实践作业

1. 为 `UserDao` 补充 `findById` ， `findAllByPage` 等方法，并完成单元测试（可用内存数据库或测试库）。
2. 将此前“图书列表展示”功能按三层架构整合到当前工程，提交包结构与类图。